# MEMORIA (Básica) TAREA DESARROLLO WEB ENTORNO SERVIDOR

**Índice**


1. **Presentación del proyecto.**
2. **Implementación del login.**
3. **Listado de productos y valoraciones.**
4. **Sistema de votación con AJAX.**
5. **Vistas con Blade.**



**Alumno:** Antonio Murillo Dávila.

**Ciclo:** Desarrollo de Aplicaciones Web.

**Módulo:** Desarrollo Web Entorno Servidor.

**Curso académico:** 2025 – 2026

IES Virgen de la Paz — Alcobendas, Madrid

1. **Presentación de proyecto.**

Para la tarea del Proyecto de DWES, la he planteado de manera que se integre con el proyecto que he presentado para el módulo de Desarrollo Web en Entorno Cliente.
Ejecutando **index.html** desde la raíz del programa, verá en el navbar las diferentes pestañas de la tarea de ese módulo junto con **la correspondiente a esta, en concreto "TAREA DWES"**. Cuando acceda, estará desplegada la tarea con el **formulario de login** y una vez logueado correctamente, se mostrará la **tabla de Productos** disponibles junto con la posibilidad de votar.

Respecto al proyecto de este módulo, lo he creado siguiendo una arquitectura en capas, con patrón DAO para acceder a los datos a través de métodos con consultas preparadas y presentando las vistas con BladeOne, instalando previamente con Composer.
El sistema de carpetas y archivos que he creado es el siguiente:

- src/Modelo/ Clases de modelo:
    - Usuario.php.
    - Producto.php.

- src/DAO/ Clases de acceso a datos:
    - UsuarioDAO.php.
    - ProductoDAO.php.
    - VotosDAO.php.

- src/Conexion.php: Clase Singleton para la conexión PDO a la base de datos.

- src/Config.php: Clase para leer variables de entorno desde el archivo .env.

- src/PintarEstrellas.php: Clase de utilidad para “Pintar” estrellas HTML.

- views/: Plantillas Blade:
    - app.blade.php.
    - login.blade.php.
    - productos.blade.php.

- js/: Archivos JavaScript:
    - validar.js (login AJAX).
    - votos.js (votos AJAX).

- login.php: Controla el login.

- productos.php: Controla el listado de productos.

- votos.php: Controlador que recibe y procesa los votos a través de AJAX.

- logout.php: Cierre de sesión.

- .env: Archivo de configuración con las credenciales de la base de datos.

## 2. Implementación del login.

Para Implementar el login, he creado un formulario que va a validar las credenciales mediante una petición AJAX antes de enviar el formulario al servidor, para así evitar que la página se vuelva a cargar.
El usuario al introducir sus credenciales, los interceptará jQuery y realizará una petición AJAX POST a login.php, que encripta la contraseña con SHA256 y busca el usuario en la base de datos usando la clase **UsuarioDAO** con el método **recuperaPorCredenciales(),** que recibirá como parámetros el nombre del usuario y su contraseña**.**

Si las credenciales son correctas, devuelve un JSON con {login:true} y jQuery redirige a **productos.php** (se guardará el objeto Usuario en al sesión de PHP)**,** en caso contrario, devuelve {login: false} y se mostrará el mensaje de error sin recargar la página de login.

En cuanto a la validación en lado del cliente, antes de realizar la petición AJAX, validar.js comprueba que los campos no estén vacíos, o que haya algún error, mostrando mensajes de error específicos para cada campo sin necesidad de contactar con el servidor. Aún así, también implementé en lado servidor esa validación en caso de que el usuario deshabilite javascript en su navegador, controlando así la validación en lado servidor.

## 3. Listado de productos y valoraciones.

La página **productos.php** es la que se va a encargar de recuperar todos los productos de la base de datos a través de **ProductoDAO** y los métodos que cree para ello:

- **recuperaTodo().**
- **recuperaPorFamilia().**

Una vez recuperados, se mostrarán en una tabla que he creado con Bootstrap y jquery, los siguientes campos para cada producto:

- Id del producto.
- Nombre.
- Precio.
- Valoración actual (número de valoraciones recibidas y media representada con estrellas visuales (llenas, medias y vacías) usando Bootstrap icons.
- Selector de puntuación (1a 5).
- Botón Votar.

Creo la clase **PintarEstrellas()** para generar el HTML de las estrellas a partir de la media calculada por **VotosDAO,** usando el método **receparMediaVotos()** que recibirá como parámetro el id del producto. Si la parte decimal de la media es mayor o igual a 0.5 se pintará media estrella.

**4. Sistema de votación con AJAX.**

**votos.js** se va a encargar de realizar la petición AJAX POST a **votos.php** con el id del producto y la puntuación que seleccione el usuario. Será el servidor quien se encargará de realizar las siguientes acciones:

- Se verifica que el usuario esté registrado o con una sesión abierta.

- A través de **VotosDAO::yaVoto(**), se comprobará si el usuario ya ha votado ese producto o no y en caso de haber votado anteriormente, se devolverá el JSON con el error “Ya has votado este producto” que se mostrará con un alert en pantalla. Si no ha votado, se mostrará el mensaje, también en un alert, con el mensaje “Voto registrado” y se guardará el voto con **VotosDAO::guardarVoto()**.

- Calcular la media con **VotosDAO::recuperaMediaVotos()** y el total con VotosDAO::contadorVotos();

- Generar el HTML de las estrellas con PintarEstrellas::render() y se devuelve JSON con la nueva valoración.

- jQuery actualiza la valoración del producto.

**5. Vistas con Blade**

He generado las vistas con el motor de plantillas BladeOne, instalandolo con Composer y he generado un patrón de herencia de plantillas de la siguiente forma:

- **app.blade.php:** es la plantilla base con el header, navbar y footer. Lo he creado con Bootstrap y jquery , de forma que siga la línea visual y poder incluirlo junto con la tarea propuesta para el módulo de Desarrollo Web Entorno Cliente.

- **login.blade.php:** que será la vista que corresponde al formulario de login y que cargará validar.js.

- **productos.blade.php:** que contiene la tabla de los diferentes productos de la base de datos y muestra los datos anteriormente descritos.